Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

ОТЧЁТ

по лабораторной работе № 3

«Проектирование схемы базы данных по описанию задачи с помощью LLM»

Вариант 3 «СТО»

по дисциплине «Автоматизация научных исследований»

Выполнили студенты группы 5040102/60201
Булыгина Екатерина Сергеевна
Дылдина Татьяна Игоревна
Крутогузенко Ксения Сергеевна

Санкт-Петербург
2026

1 ЗАДАНИЕ

Выбрано задание № 3 «СТО». Формулировка использована без изменений:

Станция по ремонту автомобилей. Таблицы: список запчастей и расходных материалов на складе (обязательные поля: цена, тип-б/у или новая, для какой модели машины подходит), список моделей машин, которые могут быть обслужены на станции, список и стоимость выполняемых работ (с указанием временного периода для периодических работ), список клиентов станции, список автомобилей обслуживаемых на станции (у одного клиента может быть несколько автомобилей, один клиент может продать автомобиль другому), список заказов (в запись входит список запчастей и выполненных работ, а также показания спидометра в момент выполнения заказа).
Требуется:

- Поддержка оформления нового заказа (с автоматическим подсчетом общей стоимости) с автоматическим оформлением нового клиента и/или автомобиля, если нужно;
- Снятие автомобиля или клиента (со всеми его автомобилями) с учета на станции;
- Фин. отчет (годовой);
- Поддержка проверки необходимости периодических работ (типа замены масла) по всем автомобилям и формирование писем (печатных) клиентам с напоминанием, что пора пригнать авто на ту или иную периодическую работу.

2 МОДЕЛИ И ИСХОДНЫЙ ЗАПРОС

Таблица 1 — Использованные модели

Модель	**Доступ**	**Условия**
Claude Sonnet 5.5	бесплатная веб-версия, модель по умолчанию	новый диалог; запрос с замечаниями — в том же диалоге
GPT	бесплатная веб-версия, модель по умолчанию	новый диалог; запрос с замечаниями — в том же диалоге
Deepseek	бесплатная веб-версия, модель по умолчанию	новый диалог; запрос с замечаниями — в том же диалоге

Всем моделям отправлен одинаковый запрос по шаблону из методических указаний:

По приведённому ниже заданию спроектируй реляционную схему базы данных.
Выдай полный код DBML для вставки в https://dbdiagram.io/.
Нужна только схема: таблицы, поля, типы данных, ключи и связи.
Все операции и отчёты из условия должны быть реализуемы на её основе.
SQL-запросы и код приложения не нужны.
Если необходимы допущения, явно отметь их в комментариях DBML.
Полное условие задания:

3 ИСХОДНЫЕ СХЕМЫ

Код схем проверен разборщиком DBML @dbml/core 12.0.0 — тем же, что используется в dbdiagram.io. Ошибок синтаксиса не обнаружено, дополнительных обращений к моделям для исправления DBML не потребовалось. Диаграммы приведены на рисунках 1, 2, 3.

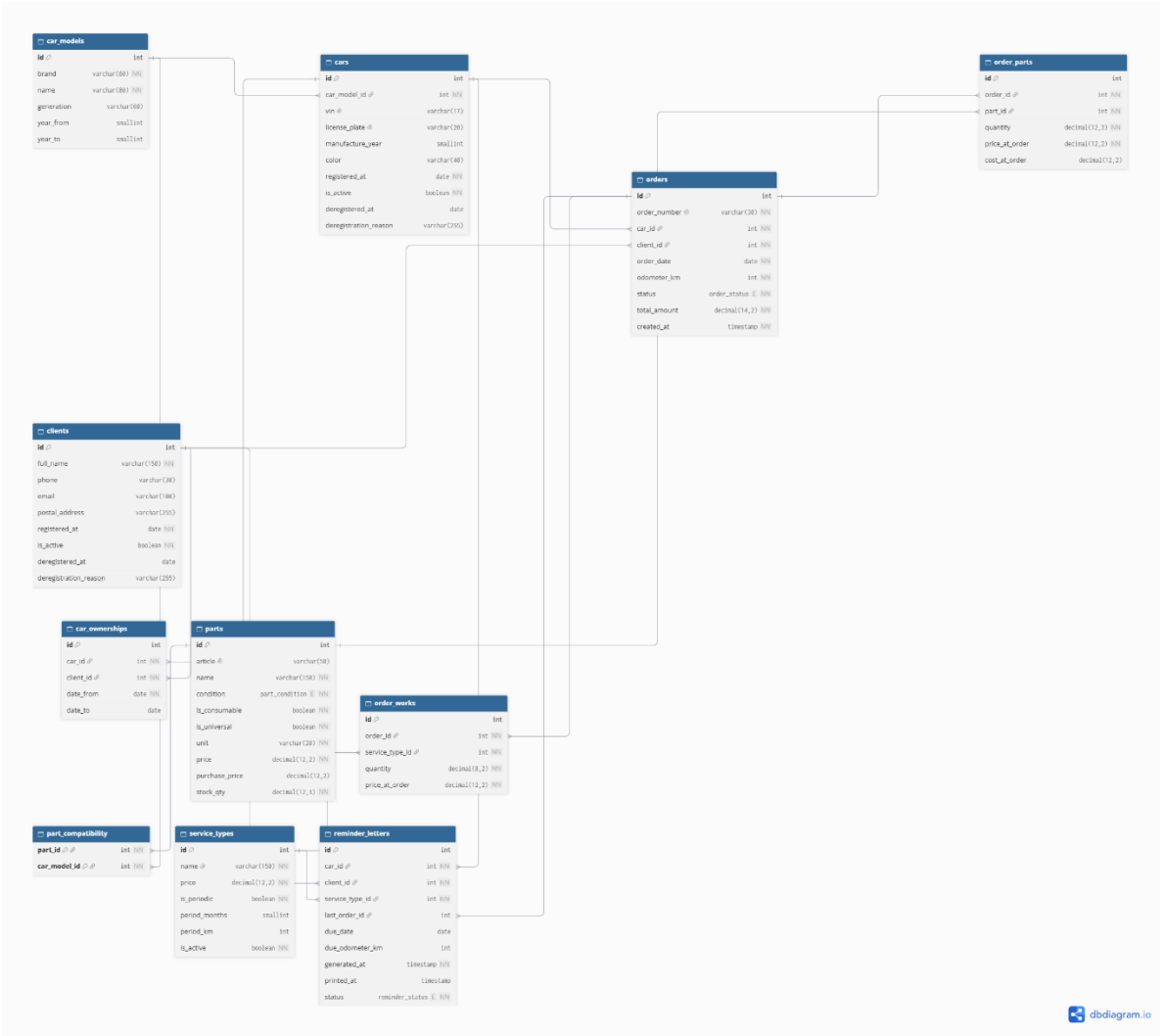


Рисунок 1 - Исходная схема Claude

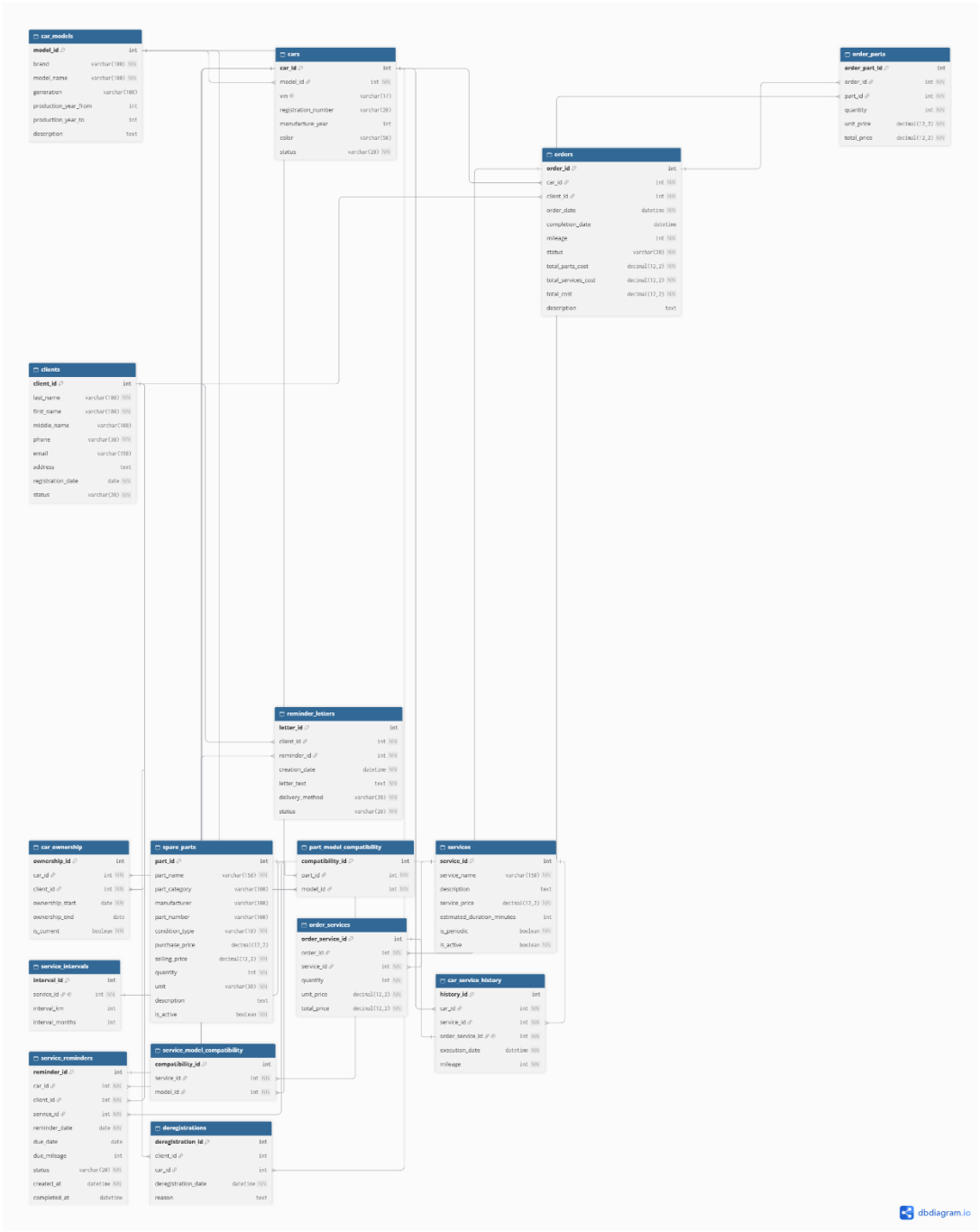


Рисунок 2 - Исходная схема GPT

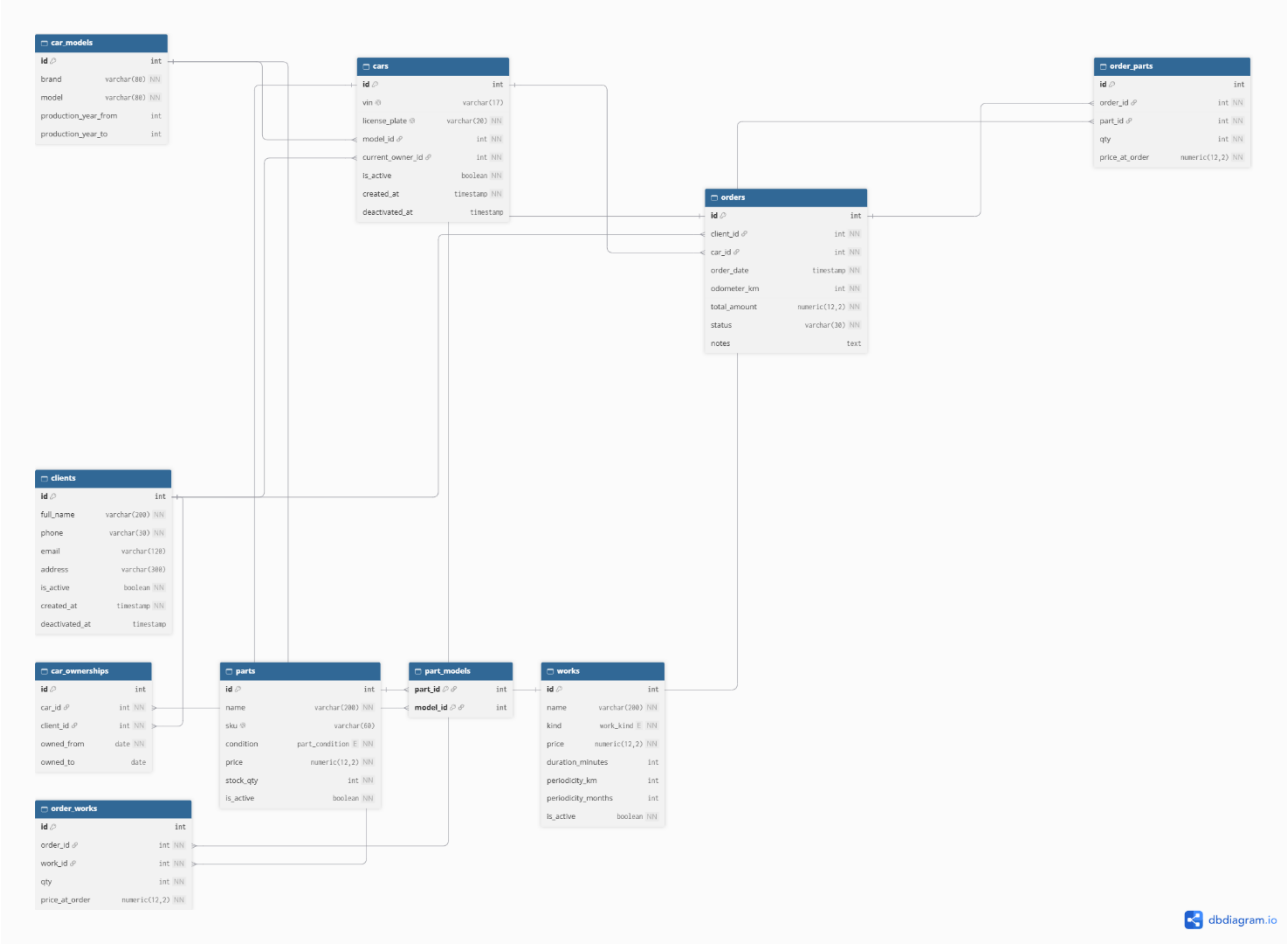


Рисунок 3 - Исходная схема Deepseek

4 КАЧЕСТВЕННЫЙ РАЗБОР ИСХОДНЫХ СХЕМ

Таблица 2 - Проблемы схемы Claude

Проблема и место в схеме	Жизненная ситуация	Последствие	Что следует изменить
`orders.client_id` дублирует `car_ownerships`, и между ними нет связи.	Автомобиль продали, а заказ оформили на бывшего владельца или на постороннего клиента.	Заказ относится к клиенту, который не владел машиной на эту дату. Отчёты по клиентам и адресат писем неверны.	Убрать `client_id` из заказа и определять клиента по владению на `order_date`. Либо сделать составной внешний ключ на `car_ownerships` (`car_id`, `client_id`) и проверять, что дата заказа попадает в период владения.
Единственность текущего владельца и корректность периодов только в комментарии. Частичный индекс `UNIQUE(car_id) WHERE date_to IS NULL` в DBML не выражается, а `date_to >= date_from` нигде не проверяется.	При оформлении продажи сбой или ошибка оператора оставляют у машины двух «текущих» владельцев или перекрывающиеся периоды.	Напоминание уходит не тому клиенту или сразу двум. История владения противоречива.	Добавить в схему и документацию частичный уникальный индекс и CHECK на даты. Для ограничения по перекрытию периодов использовать exclusion-constraint (PostgreSQL). Либо ввести `cars.current_owner_id` как быстрый указатель.
Периодичность в `service_types` (`period_months`, `period_km`) одна для всех моделей, поле `is_periodic` дублирует наличие периодов.	Замена масла у одной модели раз в 10 тыс. км, у другой раз в 15 тыс. км. Можно отметить `is_periodic = true` и не заполнить ни один период.	Напоминания приходят слишком рано или поздно. Часть записей периодических работ невозможно проверить.	Вынести интервалы в `service_intervals` (`service_type_id`, `car_model_id`, `period_months`, `period_km`). Убрать `is_periodic` или добавить CHECK, что у периодической работы задан хотя бы один период.
`reminder_letters` не защищена от дублей, и в ней нет снимка данных адресата. Статус `fulfilled` нигде не привязан к выполнившему работу заказу.	Скрипт проверки запускают дважды. Клиент меняет адрес после формирования письма, а письмо печатают позже.	Клиент получает два одинаковых письма. Печатное письмо расходится с тем, что хранится в базе, и по нему нельзя восстановить, что и куда отправили.	Уникальность активного напоминания по (`car_id`, `service_type_id`) при статусе `created`. Хранить в письме снимок адреса и ФИО. Добавить `fulfilled_order_id`. Убрать лишний `client_id` или оставить его явно как снимок.

Остаток `parts.stock_qty` хранится числом, без движений склада. `purchase_price` у детали одна.	Две партии одной детали куплены по разной цене. Два мастера одновременно списывают последний фильтр.	Остаток может уйти в минус. Себестоимость в годовом отчёте приблизительна, а расхождение с реальным складом нельзя отследить.	Добавить `stock_movements` или `part_receipts` с количеством, ценой и датой. Остаток считать по движениям, либо держать `stock_qty` с CHECK `>= 0` и обновлять только через движения.
Заказ: `total_amount` денормализован, нет оплат, скидок, дат начала и окончания. Показания спидометра не проверяются. Совместимость запчасти с машиной в `order_parts` не контролируется.	Итог не пересчитали после правки строки. Клиент платит частями или со скидкой. Ввели пробег меньше предыдущего. В заказ попала деталь от другой модели.	Годовой отчёт показывает выручку, которой не было. Расчёт периодичности по пробегу ломается из-за ошибочных данных. Заказ можно закрыть с неподходящей деталью.	Считать сумму через представление или триггер, ввести таблицу `payments` и поле скидки. Добавить даты начала и окончания работ. Контролировать монотонность пробега и совместимость детали триггером. Отчёт строить только по `status = completed`.
Снятие с учёта и поиск клиента. Флаги `is_active` и даты дублируются в `clients` и `cars`, согласованность между ними не гарантируется. У клиента нет уникального признака. `license_plate` уникален среди всех машин, включая снятые.	После снятия клиента его машина остаётся активной, а на неактивного клиента оформляют новый заказ. При автооформлении клиента создаётся дубль того же человека. Номер снятой машины достался другой машине.	Появляются «живые» машины без владельца и дубли клиентов. Нельзя зарегистрировать машину с тем же номером. Напоминания идут по снятым объектам.	Хранить статус учёта в одном месте или поддерживать связь триггером. Добавить уникальность на контакт клиента (телефон, email) или документ. Сделать уникальность номера и VIN только среди активных записей. Запретить новые заказы для неактивных клиента и машины.

Таблица 3 - Проблемы схемы GPT

Проблема и место в схеме	Жизненная ситуация	Последствие	Что следует изменить
1. Связь клиента и автомобиля (`car_ownership`**)** — отсутствует ограничение на одного текущего владельца автомобиля.	Один автомобиль одновременно оформлен на двух клиентов.	Возникают противоречия при оформлении заказов и отправке напоминаний.	Обеспечить ограничение: у автомобиля может быть только один текущий владелец.
2. Склад (`spare_parts`**)** — количество запчастей хранится одним полем	Поступление, продажа, возврат или	Невозможно восстановить остатки на определённую дату	Добавить таблицу `stock_movements` с типом операции, количеством, датой и

`quantity`, нет истории движения товаров.	списание запчастей.	и проверить историю изменения запасов.	ссылкой на заказ или поступление.
3. Периодические работы (`service_intervals`**)** — предусмотрен только один интервал для каждой работы.	Для разных моделей автомобилей замена масла требуется через разный пробег или срок.	Невозможно корректно учитывать индивидуальные регламенты обслуживания.	Добавить связь периодичности с моделью автомобиля либо модификацией автомобиля.
4. Заказы (`orders`**)** — клиент и автомобиль указываются независимо, без проверки их связи через историю владения.	Клиент оформляет заказ на автомобиль, которым уже владеет другой человек.	Заказ может быть ошибочно привязан к неправильному клиенту.	Проверять принадлежность автомобиля клиенту на дату оформления заказа через `car_ownership`.
5. Финансовая отчётность (`orders`**)** — отсутствуют сведения об оплате и возвратах денежных средств.	Клиент оплачивает заказ частями или получает возврат после завершения ремонта.	Годовой отчёт показывает начисленную стоимость заказов, а не фактические денежные поступления.	Добавить таблицу `payments` с суммой, датой, способом оплаты и статусом платежа.
6. Напоминания (`service_reminders`**,** `reminder_letters`**)** — не сохраняется информация о том, какое периодическое обслуживание уже было учтено при создании напоминания, и нет защиты от дубликатов.	Система ежедневно проверяет автомобили и повторно формирует одинаковые письма.	Клиент получает несколько одинаковых уведомлений об одной работе.	Добавить уникальный идентификатор цикла обслуживания и ограничение на повторное создание активного напоминания.
7. Снятие с учёта (`deregistrations`**)** — допускается указание одновременно клиента и автомобиля либо отсутствие обоих.	Сотрудник снимает клиента с учёта, но его автомобили продолжают числиться активными.	Возникают несогласованности статусов, истории владения и возможности оформления заказов.	Разделить снятие клиента и автомобиля либо добавить ограничение на заполнение ровно одного из двух идентификаторов и механизм снятия связанных автомобилей.

Таблица 4 - Проблемы схемы Deepseek

Проблема и место в схеме	**Жизненная ситуация**	**Последствие**	**Что следует изменить**

Глобальный UNIQUE на cars.license_plate/vin при soft-delete	Авто снято с учёта, новый клиент пригоняет машину с тем же госномером (номера переиспользуются)	INSERT падает — нельзя оформить заказ; soft-delete несовместим с UNIQUE	Частичный уникальный индекс WHERE is_active = true
Дублирование владельца: cars.current_owner_id + car_ownerships	Продажа авто от клиента A к B: обновили одно, забыли другое	Рассинхронизация «чьи авто» и истории владения; нет гарантии целостности	Убрать current_owner_id, вычислять владельца как car_ownerships WHERE owned_to IS NULL (+ частичный UNIQUE (car_id) WHERE owned_to IS NULL), либо триггер-сверка
orders.client_id не сверяется с владением на order_date	Заказ оформлен на старого владельца после продажи; или исторический заказ при смене владельца	Письма-напоминания и отчёты уходят не тому; нет проверки согласованности	Триггер/CHECK сверки orders.client_id с car_ownerships на дату заказа, либо явно задокументировать «client_id = плательщик»
order_parts/order_works без UNIQUE и без триггера на orders.total_amount	Одну запчасть добавили двумя строками; позиции изменили, total_amount не пересчитали	Двойной подсчёт, годовой отчёт врёт, расхождение с суммой позиций	UNIQUE(order_id, part_id), UNIQUE(order_id, work_id); триггеры AFTER INSERT/UPDATE/DELETE на позициях для total_amount (или VIEW вместо поля)
Периодичность работ: periodicity_* в works без привязки к модели и без CHECK	«Замена ремня ГРМ» — интервал 60/90/120 тыс. км в зависимости от модели; у kind='periodic' оба поля NULL	Проверка «пора ли» неоднозначна (км? месяцы? И/ИЛИ?), напоминания не формируются или ложные	Таблица work_intervals(work_id, model_id, interval_km, interval_months); CHECK «хотя бы одно поле NOT NULL» для periodic; явно задать логику И/ИЛИ
Нет учёта отправленных напоминаний и каскада снятия с учёта	Клиенту отправили письмо, через неделю сформировали снова — дубль; клиента деактивировали, а его авто остались is_active=true	Спам клиентам, нет аудита уведомлений; напоминания уходят «снятому» клиенту	Таблица notifications(client_id, car_id, work_id, sent_at, channel, status); триггер каскадной деактивации cars при деактивации clients
Свободные orders.status, отсутствие CHECK на цены/количество, odometer_km без монотонности, нет дат оплаты	Ввели 'открыт'/'OPEN'/'done'; отрицательная цена; опечатка в пробеге (120 000 вместо 210 000); заказ закрыт в	Нельзя фильтровать заказы, отрицательные суммы, проверка периодики по пробегу	enum order_status; CHECK price_at_order >= 0, qty > 0; CHECK/триггер монотонности odometer_km по car_id; добавить closed_at,

	январе, а отчёт по order_date	ломается, фин. отчёт расходится с бухгалтерским	paid_at/payments и определить дату годового отчёта

5 ИСПРАВЛЕНИЕ СХЕМ

На втором этапе каждой модели в том же диалоге были переданы замечания из раздела 4 с просьбой исправить схему с их учётом. Результатом стал новый вариант кода DBML для каждой модели; изменения по каждой схеме сведены в таблицы 5–7 в единой форме «было (проблема) — стало (исправление) — на что влияет».

На рисунках 4, 5, 6 представлены исправленные схемы.

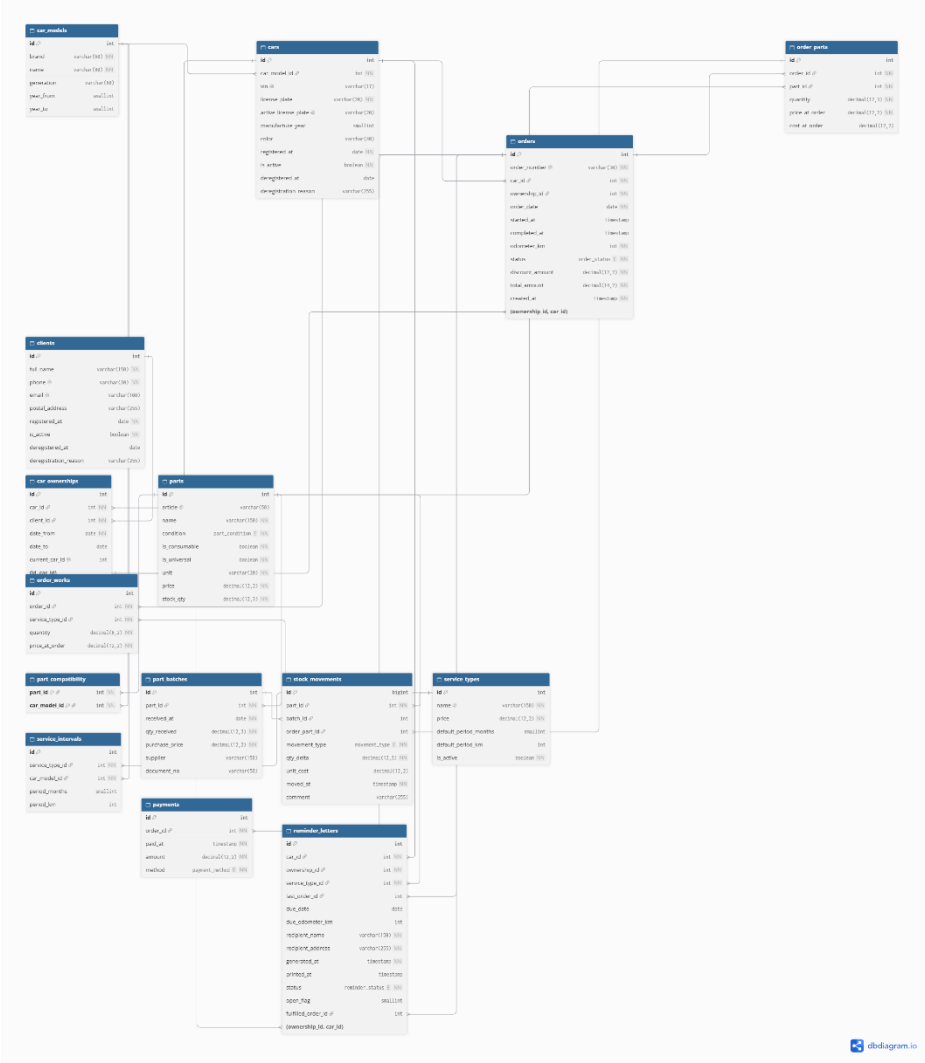


Рисунок 4 - Исправленная схема Claude

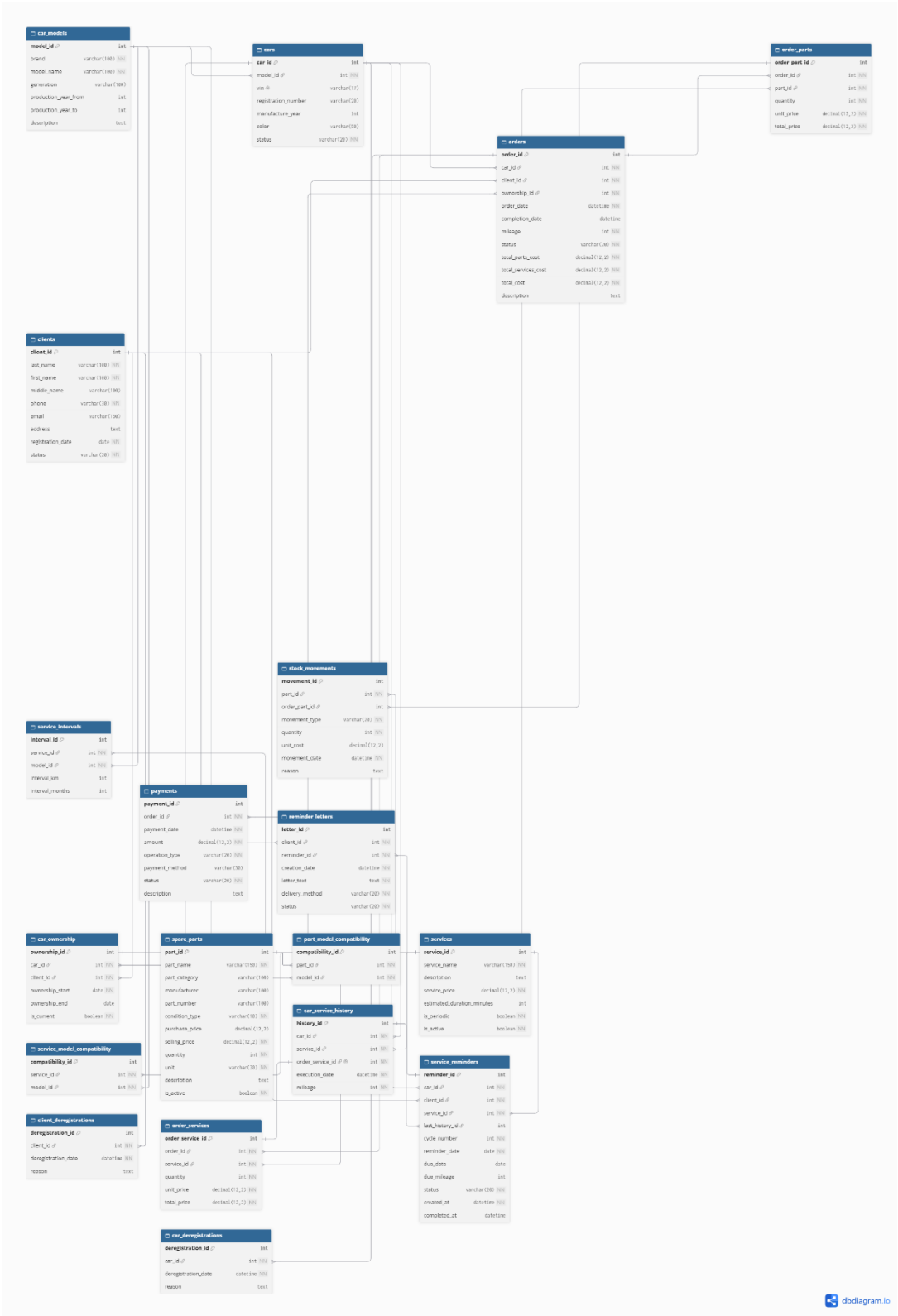


Рисунок 5 - Исправленная схема GPT

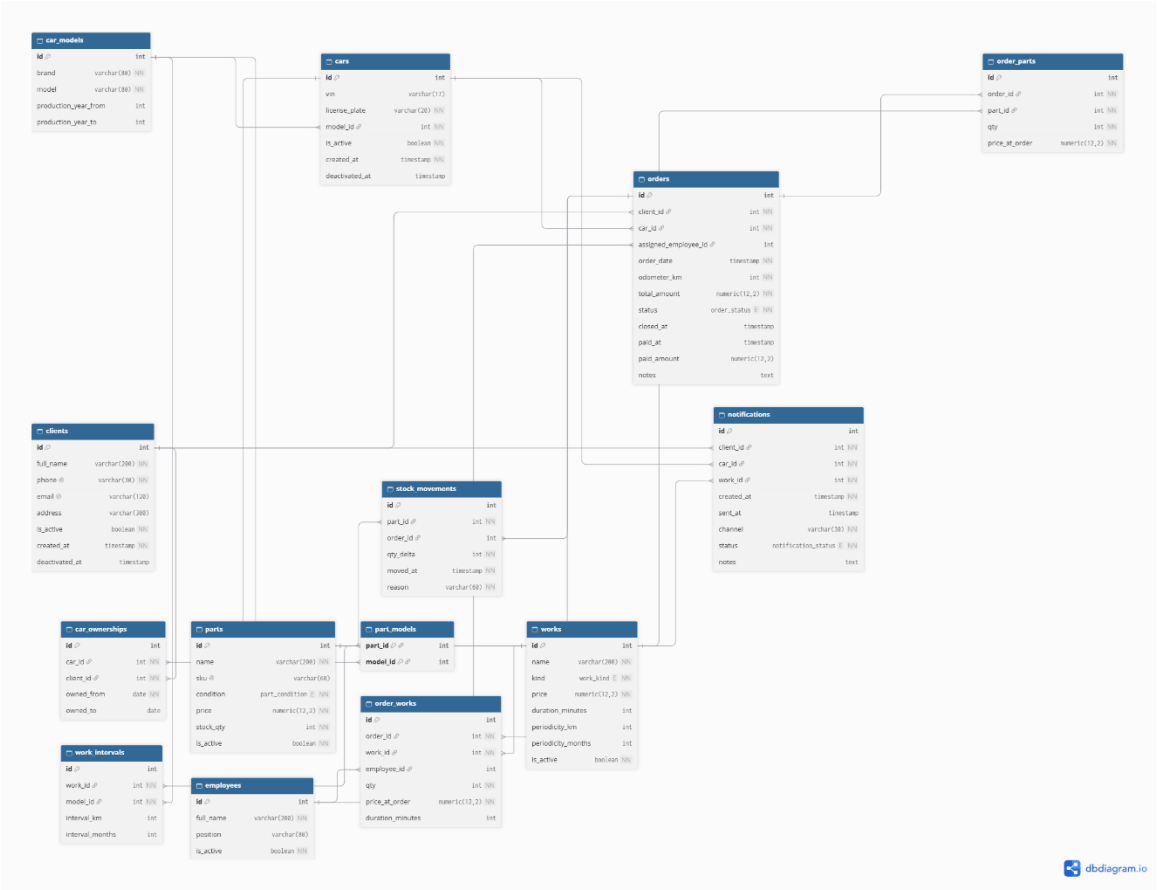


Рисунок 6 -Исправленная схема Deepseek

По сравнению с исходными вариантами исправленные схемы стали крупнее: у Claude число таблиц выросло с 11 до 15, у GPT — с 16 до 19, у Deepseek — с 10 до 14. Во всех трёх схемах появилась таблица движений склада, а остальной рост связан с выделением оплат, периодичности по моделям автомобилей, учёта писем и событий снятия с учёта.

Схема Claude (таблица 5) изменена главным образом за счёт ограничений внутри самой схемы: заказ привязан к записи владения составным ключом, единственность текущего владельца, открытого напоминания и номера активного автомобиля задана через UNIQUE и CHECK, склад разделён на партии и движения. Правила, которые DBML выразить не может, вынесены в явный список триггеров.

Таблица 5 – Итоговая таблица с изменениями схемы Claude

Было (проблема)	Стало (исправление)	На что влияет
Заказ хранил `client_id` отдельно от владения	Заказ ссылается на владение (`ownership_id`) через составной ключ с `car_id`	Корректность клиента в заказе, история после продажи авто, адресат писем

Было (проблема)	Стало (исправление)	На что влияет
Единственность текущего владельца была только в комментарии	`current_car_id` с UNIQUE, CHECK на даты, exclusion-constraint в комментарии	Целостность истории владения, отсутствие двух «текущих» владельцев
Один период работы на все модели, лишний флаг `is_periodic`	`service_intervals` по моделям, значения по умолчанию в `service_types`, флаг убран	Точность напоминаний о периодических работах
Письма без защиты от дублей и без снимка данных	Уникальность открытого напоминания, снимок ФИО и адреса, `fulfilled_order_id`	Отсутствие повторных писем, возможность восстановить, что и кому отправили
Остаток числом, одна закупочная цена на деталь	`part_batches` и `stock_movements`, `stock_qty` как кэш с CHECK `>= 0`	Учёт склада, себестоимость и прибыль в годовом отчёте
Нет оплат, скидок и дат выполнения заказа	`payments`, `discount_amount`, `started_at`, `completed_at`, CHECK-проверки	Точность выручки и поступлений денег, работа с частичной оплатой
Статус снятия с учёта дублировался без проверок, номер машины уникален глобально, клиентов можно задвоить	CHECK на статус и дату, `active_license_plate`, уникальные телефон и email	Снятие с учёта, отсутствие дублей клиентов, повторная регистрация номера
Проверки пробега, совместимости деталей и активности клиента нигде не описаны	Вынесены в явный список триггеров (допущение 10)	Качество данных заказа и надёжность расчёта периодичности

В схеме GPT (таблица 6) основной упор сделан на добавление новых сущностей: появились история движения склада, платежи с возвратами и отдельные таблицы снятия с учёта клиентов и автомобилей, а периодичность работ привязана к модели автомобиля. CHECK и триггеры в таблице 6 не упоминаются.

Таблица 6 – Итоговая таблица с изменениями схемы GPT

Было (проблема)	Стало (исправление)	На что влияет
Не было ограничения на количество владельцев автомобиля	Добавлена история владения с привязкой к заказам	Исключает противоречия при продаже автомобилей и оформлении заказов
Хранился только текущий остаток запчастей	Добавлена история движения складских запасов	Позволяет контролировать поступления, списания, возвраты и остатки
Периодичность работ задавалась без учёта модели	Добавлена периодичность отдельно для каждой модели автомобиля	Повышает точность контроля технического обслуживания

Заказ не проверял принадлежность автомобиля клиенту	Добавлена связь заказа с историей владения	Предотвращает оформление заказа от имени неправильного владельца
Финансовый учёт основывался только на стоимости заказов	Добавлена таблица платежей и возвратов	Позволяет формировать более точную годовую финансовую отчётность
Не было защиты от повторных напоминаний	Добавлен номер цикла обслуживания	Предотвращает дублирование писем клиентам
Снятие с учёта клиента и автомобиля было объединено	Добавлены отдельные таблицы снятия с учёта	Упрощает управление статусами клиентов и автомобилей

Исправления Deepseek (таблица 7) самые многочисленные — 14 пунктов при 7 замечаниях в таблице 4 — и опираются на средства СУБД: частичные уникальные индексы, EXCLUDE, триггеры, перечисления и CHECK. Кроме того, модель добавила сущность исполнителей работ (employees), которой в условии задания нет.

Таблица 7 – Итоговая таблица с изменениями схемы Deepseek

Было (проблема)	**Стало (исправление)**	**На что влияет**
cars.current_owner_id + car_ownerships — два источника истины о владельце	Владелец только из car_ownerships WHERE owned_to IS NULL + частичный UNIQUE(car_id) WHERE owned_to IS NULL и EXCLUDE на непересечение периодов	Корректность «чьи авто», история продаж, письма и отчёты по клиенту
Глобальный UNIQUE на license_plate/vin при soft-delete	Частичные уникальные индексы WHERE is_active = true	Возможность повторно принять авто с тем же номером/VIN после снятия с учёта
orders.status — свободный varchar	enum order_status {open, in_progress, closed, cancelled}	Надёжная фильтрация заказов, отчёты, напоминания
orders.total_amount — денормализация без поддержки	Триггеры AFTER INSERT/UPDATE/DELETE на order_parts/order_works	Годовой фин. отчёт и чек совпадают с суммой позиций
Нет UNIQUE в позициях заказа	UNIQUE(order_id, part_id), UNIQUE(order_id, work_id)	Исключение двойного подсчёта, дублей в чеке
orders.odometer_km без проверки	CHECK/триггер монотонности по car_id относительно предыдущего заказа	Корректная проверка периодических работ «по пробегу»
Периодичность только в works.periodicity_*	Таблица work_intervals(work_id, model_id, interval_km, interval_months) + CHECK «хотя бы одно поле NOT NULL»	Точная проверка «пора ли» для конкретной модели; корректные письма-напоминания
Нет учёта отправленных писем	Таблица notifications(client_id, car_id, work_id, sent_at, status)	Без спама клиентам, аудит уведомлений

Было (проблема)	**Стало (исправление)**	**На что влияет**
Снятие клиента не каскадируется на авто	Триггер каскадной деактивации cars + CHECK is_active ↔ deactivated_at	Авто «снятого» клиента не попадают в напоминания и отчёты
orders.client_id не сверяется с владением	Триггер/документированное правило: client_id — заказчик на order_date, сверка с car_ownerships	Правильный адресат писем и отчётов по заказам
Годовой отчёт по order_date без даты закрытия	Поля closed_at, paid_at, paid_amount; отчёт по closed_at	Совпадение фин. отчёта с бухгалтерским учётом
parts.stock_qty без учёта движений	Таблица stock_movements(qty_delta, reason, order_id) + триггер на stock_qty	Корректные остатки склада, планирование закупок
Нет исполнителей работ	employees + order_works.employee_id, снимок duration_minutes	Отчёты по загрузке/выработке, ответственность за заказ
Нет CHECK на цены/количество	CHECK price >= 0, qty > 0, stock_qty >= 0, odometer_km >= 0	Отсутствие отрицательных сумм и абсурдных значений

Таким образом, все три модели за одну итерацию устранили основные замечания, но разными способами: Claude усилил ограничения внутри схемы, GPT расширил набор сущностей, Deepseek сделал ставку на механизмы СУБД, не отображаемые на диаграмме. Сопоставление этих подходов приведено в разделе 6.

6 СРАВНЕНИЕ МОДЕЛЕЙ

Сравнение выполнено по исправленным схемам, так как они отражают итоговый результат работы каждой модели после одной итерации «схема — замечания — исправление». Количественные показатели приведены в таблице 8, сопоставление по ключевым вопросам задания — в таблице 9.

Таблица 8 – Количественные показатели схем

Показатель	Claude	GPT	Deepseek
Таблиц в исходной схеме	11	16	10
Таблиц в исправленной схеме	15	19	14
Замечаний в разделе 4	7	7	7
Пунктов в итоговой таблице раздела 5	8	7	14
Новые таблицы	part_batches, stock_movements, service_intervals, payments	stock_movements, payments, client_deregistrations, car_deregistrations (вместо deregistrations)	stock_movements, work_intervals, notifications, employees

Исходные схемы различались по размеру (от 10 до 16 таблиц), а число замечаний оказалось одинаковым — по 7 на схему. После исправления каждая схема выросла на 3–4 таблицы. Самый подробный перечень изменений у Deepseek (14 пунктов), но большая часть из них — точечные ограничения, поля и триггеры, а не новые сущности.

Таблица 9 – Сравнение исправленных схем по ключевым вопросам

Вопрос	Claude	GPT	Deepseek
Связь заказа с владением	ownership_id вместо client_id, составной ключ (ownership_id, car_id)	Добавлен ownership_id, client_id в заказе сохранён	client_id сохранён, сверка с car_ownerships — триггером или документированным правилом

Вопрос	Claude	GPT	Deepseek
Единственный текущий владелец	current_car_id с UNIQUE и CHECK; непересечение периодов — EXCLUDE в комментарии	Флаг is_current в car_ownership; способ контроля в таблице 6 не указан	current_owner_id удалён; частичный UNIQUE и EXCLUDE на непересечение периодов
Склад и себестоимость	part_batches (закупочная цена партии) и stock_movements; cost_at_order в строке заказа	stock_movements с unit_cost; purchase_price в spare_parts	stock_movements с триггером на stock_qty; закупочная цена не хранится
Оплаты и финансовый отчёт	payments (оплата частями), скидка, отчёт по завершённым заказам; возвраты не поддержаны	payments с типом операции и статусом (оплаты и возвраты); отдельные итоги по работам и запчастям	closed_at, paid_at, paid_amount в заказе; оплата частями не поддержана
Периодические работы	service_intervals (работа × модель), значения по умолчанию в service_types, CHECK на наличие периода; is_periodic удалён	service_intervals с model_id; флаг is_periodic в services сохранён	work_intervals (работа × модель), CHECK «хотя бы одно поле NOT NULL», логика И/ИЛИ задаётся явно
Письма-напоминания	Уникальность открытого напоминания, снимок ФИО и адреса, ссылка на заказ, закрывший напоминание	cycle_number в service_reminders; reminder_letters хранит letter_text и delivery_method	notifications (создание, отправка, канал, статус); поля для текста письма и снимка адреса нет
Снятие с учёта	is_active и дата снятия с CHECK, номер уникален среди активных; каскад выполняет приложение в транзакции	Отдельные таблицы client_deregistrations и car_deregistrations; каскад по автомобилям клиента в таблице 6 не описан	is_active и deactivated_at с CHECK, триггер каскадной деактивации автомобилей
Проверки данных	Около 30 CHECK и enum в коде DBML; пробег, совместимость деталей и итог заказа — триггеры из допущения 10	CHECK и триггеры в таблице 6 не упомянуты; статусы на диаграмме — varchar	enum статусов, CHECK на цены, количества и пробег, UNIQUE в позициях заказа, триггеры пересчёта итога

По перечню изменений в таблицах 5–7 схема Claude опирается на ограничения, которые выражаются в самом DBML: составной внешний ключ заказа и владения, UNIQUE, CHECK, перечисления. У Deepseek ключевые механизмы — частичные уникальные индексы, EXCLUDE и триггеры — DBML не выражает, поэтому на диаграмме они не видны и требуют отдельной реализации в СУБД. Схема GPT богаче всех сущностями (19 таблиц): платежи с возвратами, история снятия с учёта, хранение текста письма и истории выполнения работ, однако ограничения целостности в перечне изменений не названы.

Слабые места остались у каждой схемы. В схеме Claude платежи не поддерживают возвраты, а отсутствие UNIQUE в позициях заказа допускает внесение одной детали несколькими строками. В схеме GPT на диаграмме client_id и ownership_id присутствуют в заказе одновременно, поэтому их согласованность из перечня изменений не следует; статусы остались свободными строками, а история выполнения работ хранит дату и пробег, уже имеющиеся в заказе. В схеме Deepseek не хранится закупочная цена, поэтому годовой отчёт ограничен выручкой и итоговой оплатой заказа; оплата частями не поддерживается; client_id в заказе сохранён и сверяется только правилом или триггером; таблица исполнителей выходит за рамки условия.

Четыре требования задания реализуемы на всех трёх схемах: итог заказа вычисляется по позициям, новые клиент и автомобиль создаются вставкой записей перед заказом, снятие с учёта выражено статусом и датой, а письма формируются по периодичности, заданной для модели автомобиля. Различия лежат в надёжности и точности. Годовой отчёт с прибылью наиболее полно поддерживает схема Claude, где себестоимость фиксируется в строке заказа по партиям; отчёт по фактическим поступлениям и возвратам — схема GPT; только по выручке и итоговой оплате — схема Deepseek. Защита от повторных писем и снимок данных адресата заложены в схеме Claude, текст письма хранит схема GPT.

Ограничения сравнения. Каждая модель запускалась один раз в бесплатной версии с настройками по умолчанию, поэтому повторный запуск может дать другой результат. Разбор в разделе 4 качественный, число замечаний не является мерой качества схемы. Для GPT и Deepseek сопоставление опирается на диаграммы и таблицы изменений; частичные индексы, EXCLUDE и триггеры DBML не отображает, поэтому их наличие оценивалось по описанию. Загрузка схем в СУБД и проверка на тестовых данных не выполнялась.

7 ВЫВОДЫ

В ходе работы три языковые модели (Claude Sonnet 5.5, GPT и Deepseek) по одному запросу спроектировали реляционную схему базы данных станции технического обслуживания; схемы были проанализированы, исправлены с учётом замечаний и сопоставлены. По результатам работы сделаны следующие выводы.

1. Все три модели по одинаковому запросу выдали синтаксически корректный код DBML, принятый разборщиком @dbml/core 12.0.0 без доработок. Исходные схемы охватывали основные сущности задания: запчасти, модели и автомобили, работы, клиентов, заказы и историю владения; учёт писем-напоминаний в исходной схеме Deepseek отсутствовал.

2. Синтаксическая корректность не гарантировала целостности данных: в каждой исходной схеме найдено по 7 замечаний. У всех трёх моделей повторились пять проблем: не гарантирована единственность текущего владельца автомобиля, заказ не сверяется с историей владения, периодичность работ не зависит от модели автомобиля, нет защиты от повторных писем, снятие клиента не распространяется на его автомобили. Это правила, связывающие сущности во времени, а не перечень таблиц и полей, и именно в них модели ошибались чаще всего.

3. Цикл «исходная схема — замечания — исправление» за одну итерацию устранил основные замечания у всех моделей. Число таблиц выросло с 11 до 15 у Claude, с 16 до 19 у GPT и с 10 до 14 у Deepseek; во всех трёх схемах появилась таблица движений склада.

4. Подходы к исправлению различаются. Claude закрепил правила в структуре схемы (составной внешний ключ заказа и владения, UNIQUE, CHECK, партии склада). GPT расширил набор сущностей (платежи и возвраты, отдельные таблицы снятия с учёта). Deepseek опирался на механизмы СУБД, которые DBML не выражает (частичные индексы, EXCLUDE, триггеры), и добавил сущность, не требуемую условием, — исполнителей работ.

5. Единого лидера по всем критериям нет. Схема Claude сильнее в связи заказа с владением, учёте себестоимости по партиям и защите писем от дублей; схема GPT — в финансовом учёте (оплаты и возвраты) и истории снятия с учёта; схема Deepseek — в полноте проверок данных (перечисления, CHECK, UNIQUE в позициях заказа, контроль пробега). У каждой схемы остались слабые места, перечисленные в разделе 6.

6. Часть требований целостности DBML выразить не может: частичные уникальные индексы, непересечение периодов владения, неубывание пробега, совместимость детали с моделью автомобиля, пересчёт итога заказа. Такие правила приходится описывать комментариями или триггерами, поэтому перед практическим применением схемы их нужно реализовать в СУБД и проверить.

7. Языковые модели пригодны для быстрого получения рабочего черновика схемы и для критического разбора по заданному шаблону, но структурные ошибки целостности синтаксической проверкой не выявляются и требуют контроля человеком. Практичная последовательность работы: черновик схемы, разбор по жизненным ситуациям, исправление, проверка разборщиком DBML, затем загрузка в СУБД и проверка сценариев — продажа автомобиля, повторный запуск проверки периодических работ, снятие клиента вместе с автомобилями.